SEED Design Doc

Goal (User Perspective)

For any record-level data management task, the user can quickly build an executable program with only a few natural language descriptions.

For example, for an entity resolution task on a specfic dataset about products. The user fills in the following `json` config:

```
{
    "task_name": "entity_resolution",
    "task_desc": "Given two products, determine whether they are the same product.",
    "inputs": [
        {
            "name": "entity1",
            "type": "dict",
            "desc": "It contains three attributes: `title`, `manufacturer`, `price`. `title` and `manufacturer` are str
        },
        {
            "name": "entity2",
            "type": "dict",
            "desc": "Same as entity1."
        }
    ],
    "outputs": [
        {
            "name": "is_same",
            "type": "bool",
            "desc": "0 if the two product are not identical, 1 of the two products are identical."
        }
    ],
    "tools": [
    ],
    "examples": "./data/samples.jsonl",
    "evaluation_metric": "f1",
    "activate_cache": true,
    "activate_model": true,
    "activate_codeg": false,
    "activate_tools": false,
    "activate_llmqa": true
}
```

Then the config is compiled to a black box API `entity_resolution(entity1, entity2)`, which user can import and directly put to use. The data should be loaded as a list of dictionaries with each dictionary corresponding to a record-level instance:

```
dataset = LoadJson("./data/test.jsonl", backend='jsonl')
# [{'entity1': {'title': ..., 'manufacturer': ..., 'title': ...}, 'entity2': {'title': ..., 'manufacturer': ..., 'title
```

Then each record can be processed separately:

```python
from entity_resolution import entity_resolution
results = [
    entity_resolution(entity1=record['entity1'], entity2=record['entity2'])
    for record in dataset
]
# [{'is_same': 1}, ...]
```

or via a batched service:

```python
from entity_resolution import entity_resolution_service
er = entity_resolution_service()
for record in dataset:
    er.process(entity1=record['entity1'], entity2=record['entity2'])
results = er.flush()
# [{'is_same': 1}, ...]
```

or more simply:

```python
from entity_resolution import entity_resolution_service
er = entity_resolution_service()
results = er.map(dataset, flush=True)
# [{'is_same': 1}, ...]
```

When providing `examples` and `evaluation_metric`, a profile will also be generated for different hyperparameters:

```
cache:
    d       f1      llm%
    ...     ...     ...
    ...     ...     ...
model:
    c       f1      llm%
    ...     ...     ...
    ...     ...     ...
```

This enables the user to further customize each agent in the `json` config file:

```json
{
    ...,
    "activate_cache": true,
    "cache_d": ...,
    "activate_model": true,
    "model_c": ...,
    ...
}
```

System Perspective

Data Routing

The system consists of multiple agents (cache, model, codegen, tools, etc.). Each agent aims to provide an API similar to that of the final API.

For example, in the forementioned example `entity_resolution(entity1, entity2)`, the cache agent will be another black box `entity_resolution_cache(entity1, entity2)`. Then for the overall program, the implementation would be roughly equivalent to:

```
def entity_resolution(entity1, entity2):
    cache_result = entity_resolution_cache(entity1, entity2)
    if cache_result is not None: return cache_result
    model_result = entity_resolution_model(entity1, entity2)
    if model_result is not None: return model_result
    llmqa_result = entity_resolution_llmqa(entity1, entity2)
    if llmqa_result is not None: return llmqa_result
    return None
```

To support flexibility of using different agents. The system creates a workspace for each data record, starting with the inputs only:

```
{
    "inputs": {
        "entity1": ...,
        "entity2": ...,
    }
}
```

Then after running each agent, a new field is added to the workspace:

```
{
    "inputs": {
        "entity1": ...,
        "entity2": ...,
    },
    "cache_result": {
        "outputs": {
            "is_same": ...,
        },
        "candidates": ...,
        "confidence": ...,
    },
    ...,
}
```

Which allows further complications to other agents `entity_resolution_llmqa(entity1, entity2, cache_result=cache_result, model_result=model_result, advice=advic_result, ...)`. Therefore, the execution order of the agents should be fixed or configured in the `json` config file.

A later agent would use the response of previous agents only if the result is present and well-organized. Therefore the agents are modularized and the failure of one agent does not influence others. The same applies to the overall API.

Agent Implementation

Each agent should be defined by a `compile` function and an `update` function.

After running the `compile` function, the API of the agent should be generated and ready to be used. For example, in the forementioned entity resolution example, `cache.compile(config)` generates a sub-folder `entity_resolution_cache` which allows `from entity_resolution_cache import entity_resolution_cache`. A set of code templates can be referenced in the compilation part, to alleviate prompt engineering burdens.

For the `update` function, a data record with label (or pseudo label) is attached to allow the agent to improve itself dynamically `cache.update(config, record)`. The returned result of the overall API `entity_resolution(entity1, entity2)` will be used to update each individual agent.

To support profiling, the hyperparameters and hyperparameter ranges should be defined.

Agent: Cache

```
.
└── /entity_resolution/
    ├── apis/
    |   ├── entity_resolution_cache/
    |   |   ├── __init__.py
    |   |   ├── ckpt/
    |   |   ├── index/
    |   |   ├── data/
    |   |   └── ...
    |   └── ...
    └── ...
```

During compilation, a frozen checkpoint, a data storage, and an index is initialized (can be configured in the `json` config file).
For each update, the new record is stored in `data/` and the `index/` is updated.

Agent: Model

```
.
└── /entity_resolution/
    ├── apis/
    |   ├── entity_resolution_model/
    |   |   ├── __init__.py
    |   |   ├── ckpt/
    |   |   ├── data/
    |   |   └── ...
    |   └── ...
    └── ...
```

During compilation, a checkpoint, and a data storage is initialized (can be configured in the `json` config file).
For each update, the new record is stored in `data/`. When accumulating enough `data/`, the model is fine-tuned and the `ckpt/` is updated.

Agent: Code Ensemble

```
.
└── /entity_resolution/
    ├── apis/
    |   ├── entity_resolution_codeg/
    |   |   ├── __init__.py
    |   |   ├── testcases/
    |   |   ├── snippets/
    |   |   ├── evaluations/
    |   |   └── ...
    |   └── ...
    └── ...
```

During compilation, a series of snippets are generated and evaluated. The ensemble API (in `__init__.py`) will also be modified according to the evaluation result. To keep all code versions, snippets will never be abandoned, whcile the set of active snippets should be kept track of.
Generally, there is no update to the code generation part. However, user may edit test cases, snippets and explicitly ask for a re-compile.
It is worth noticing that the evaluation may require `from entity_resolution_llmqa import entity_resolution_llmqa`, which means they should be compiled before it.

Agent: Advisor

```
.
└── /entity_resolution/
    ├── apis/
    |   ├── entity_resolution_advic/
    |   |   ├── __init__.py
    |   |   └── ...
    |   └── ...
    └── ...
```

The advice generation does not require complex compilation or update as the implementation is almost always the same (querying an LLM for advice).
Notice that this advice is record-level, which is different from the advice used for code generation.

Agent: Tools

```
.
└── /entity_resolution/
    ├── apis/
    |   ├── entity_resolution_tools/
    |   |   ├── __init__.py
    |   |   └── ...
    |   └── ...
    └── ...
```

The tools that are allowed to use are configured in the `json` config file. Each tool should also be provided in the `apis/` folder as an individual agent.
The tools invocation does not require complex compilation or update as the implementation is almost always the same (querying an LLM iteratively).

Agent: LLM

```
.
└── /entity_resolution/
    ├── apis/
    |   ├── entity_resolution_llmqa/
    |   |   ├── __init__.py
    |   |   └── ...
    |   └── ...
    └── ...
```

The llm query does not require complex compilation or update as the implementation is almost always the same (querying an LLM directly).

System Utils Implementation

- [ ] File operations
- [ ] Config load & save
- [ ] Workspace load & save
- [ ] Custom string formatting (to avoid conflict with escape characters)
- [ ] Dynamic Python value eval (to align LLM outputs with the system)
- [ ] `LLMQuery(messages) -> response`
- [ ] `LLMTools(messages, tools) -> response`
- [ ] Response-to-code parsing (response -> code string)
- [ ] Code-to-api parsing (code string -> code api)
- [ ] Automatic Model load & training (to align different models in huggingface)
- [ ] Sandbox with exception handle

Features

(*Italic* for advanced features to be added)

- [ ] LLM query
  - [ ] Exact cache
  - [ ] Keeping all chat history
  - [ ] *Retrieval-Augmented Generation (RAG)*
- [ ] Code generation
  - [ ] Advisor in code generation
  - [ ] Verfication in code generation
  - [ ] Advisor in code fixing
  - [ ] Code filter
  - [ ] Code ensemble
  - [ ] *Keeping all code versions*
  - [ ] *Manual intervention*
- [ ] Model generation
  - [ ] Custom model
  - [ ] Data accumulation
  - [ ] Dynamic update
  - [ ] Distillation
  - [ ] *Automatic checkpoint selection*
  - [ ] *Support local LLM (7B)*
- [ ] Query batching

- ☐ Asynchronous batching
- ☐ Iterative tools invocation
  - ☐ *Generated code serves as a tool*
  - ☐ *Generated model serves as a tool*
  - ☐ *Advisor in tools invocation: interaction template*
  - ☐ *Tools invocation result as advice to aid code generation*
  - ☐ *Tools proposal*
- ☐ *Profile: ratio of each local agent, execution time*
- ☐ Generalizability for new templates
- ☐ *Generalizability for new local agents*
- ☐ *Meta-Manager: decide what local agents to use*
- ☐ *New GPT-4 API: function call & assistant*
- ☐ *CoT anywhere*